

The Missing Premise

ISE-2 Technical Summary Report · Discrete Mathematics (7MA206) · S.Y. B.Tech. (IT)

1. Project details

Title	The Missing Premise
Team ID	TeenTitans
Track	A: Gamified Application
Unit and topic	Unit I, Logic & Proof Techniques: propositional logic, truth tables, rules of inference, resolution
GitHub	https://github.com/aritradas2005/theMissingPremise
Live game	https://themissingpremise.streamlit.app/
Platform, libraries	Python 3.10+, Streamlit (web GUI), pytest (tests)

2. Team members

Roll no.	Name	Role	Share
256109007	Aritra Das	Logic core, game logic, cases	34%
256109015	Prathmesh Khare	Inference module	33%
256109003	Ganesh Chavan	Screens and artwork	33%
Total			100%

3. Problem statement and objective

The Missing Premise is a detective game for students learning propositional logic. Each case is an argument: witness statements and evidence are the premises, and the accusation is the conclusion. The statements alone never prove the conclusion, so the player must find the piece of evidence that closes the gap (the missing premise) and then prove the conclusion one checked step at a time with rules of inference. Any argument can also be typed in and analysed or played. The game demonstrates truth tables, equivalence, validity, rules of inference, resolution, and proof by direct derivation, contradiction, contraposition, exhaustion and counterexample.

4. Mathematical formulation

Formula. A statement built from letters (atoms) $P, Q, \dots$ joined by $\neg$ not, $\wedge$ and, $\vee$ or, $\rightarrow$ if...then, $\leftrightarrow$ if and only if. $\neg$ binds tightest, then $\wedge, \vee, \rightarrow, \leftrightarrow$.

Truth table. One row for every way of making the atoms true or false: with n atoms, 2^n rows. A formula true in every row is a *tautology*, false in every row a *contradiction*, otherwise a *contingency*. Two formulas are equivalent, $A \equiv B$, when they agree in every row.

Valid argument. Premises and a conclusion, written $P_1, \dots, P_k \therefore C$. It is valid when C is true in every row where all the premises are true; that is, when $(P_1 \wedge \dots \wedge P_k) \rightarrow C$ is a tautology. A row where all the premises are true is a *critical row*. A critical row where C is false is a *counterexample*.

Missing premise. When an argument is invalid, a piece of evidence M is the missing premise if adding it leaves no counterexample ($P_1, \dots, P_k, M \therefore C$ is valid) while the premises can still all be true together (they stay *consistent*).

Rules of inference. Each gives a new line from lines already on the board ($\vdash$ reads "gives").

Modus Ponens	$P \rightarrow Q, P \vdash Q$	Modus Tollens	$P \rightarrow Q, \neg Q \vdash \neg P$
Hypothetical Syllogism	$P \rightarrow Q, Q \rightarrow R \vdash P \rightarrow R$	Disjunctive Syllogism	$P \vee Q, \neg P \vdash Q$
Resolution	$P \vee Q, \neg P \vee R \vdash Q \vee R$	Contraposition	$P \rightarrow Q \vdash \neg Q \rightarrow \neg P$
Addition	$P \vdash P \vee Q$	Simplification	$P \wedge Q \vdash P$
Conjunction	$P, Q \vdash P \wedge Q$		

Resolution check. Every verdict is confirmed a second way: the premises and $\neg C$ are written as clauses (ORs of letters) and opposite letters are cancelled pair by pair. Reaching the empty clause $\square$ means C follows.

Worked example (Case 1). W : the thief came in through the window; G : he crossed the garden; B : the dog barked. Statements: $W \rightarrow G$ and $G \rightarrow B$. To prove: $\neg W$. The critical rows (both statements true) are:

W	G	B	$W \rightarrow G$	$G \rightarrow B$	$\neg W$
T	T	T	T	T	F
F	T	T	T	T	T
F	F	T	T	T	T
F	F	F	T	T	T

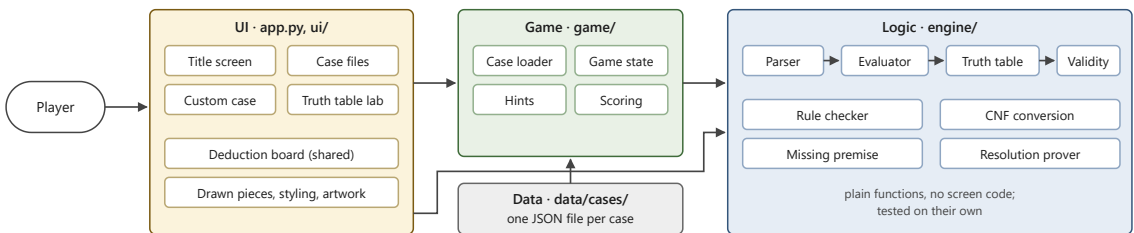
counterexample

There is a counterexample, so the statements alone do not prove $\neg W$. The evidence $\neg B$ (the dog did not bark) rules out the first three rows, and $\neg W$ is true in the one that is left: $\neg B$ is the missing premise. Proof: $G \rightarrow B, \neg B \vdash \neg G$, then $W \rightarrow G, \neg G \vdash \neg W$, both by Modus Tollens.

Algorithm.

1. Parse each typed formula into a tree; a mistake is reported with its position.
2. List the n atoms and all 2^n rows.
3. Work out every premise and the conclusion on each row.
4. Mark the critical rows. The argument is valid if none has a false conclusion; otherwise the counterexamples are shown.
5. If it is invalid, try each piece of evidence as an extra premise. It is the missing premise if no counterexample remains and a critical row is left.
6. Check each step the player enters against the chosen rule: accept it as a numbered line, or reject it with a reason. Solved when C is derived (direct proof), or when a formula and its negation are derived after assuming the opposite of C (contradiction).

5. Software architecture



UI takes typed formulas and button presses and draws tables, proofs and the detective; it has no logic. **Game** loads and checks case files, holds the proof, evidence and mistakes of the case in play, and computes hints and the score. **Logic** is all the mathematics. **Data** files never say which evidence is the missing premise; the logic works it out, so a typed case plays exactly like a written one.

6. Key UI screenshots

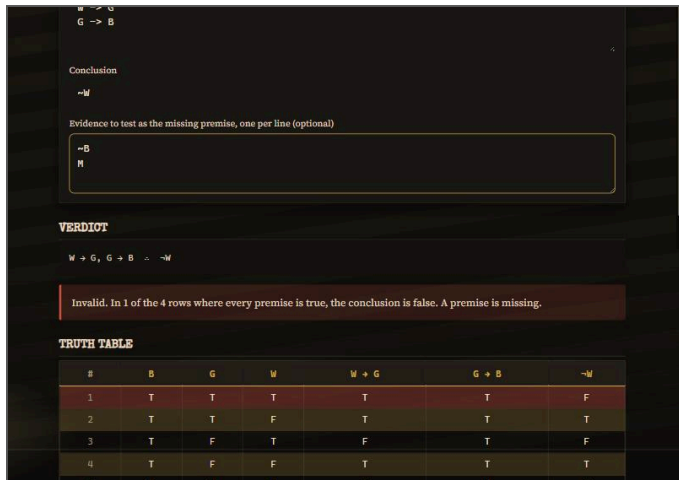


Figure 1. Input screen (custom case). Premises, a conclusion and evidence typed by the user. The verdict is "invalid", and the truth table marks the counterexample row in red and the other critical rows in gold.

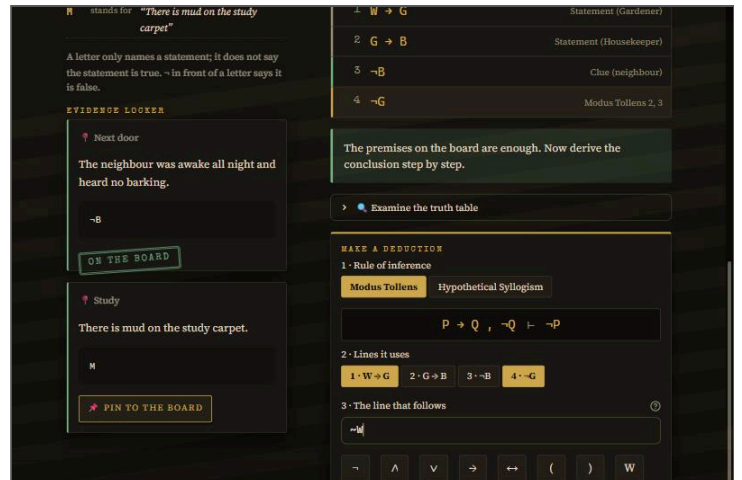


Figure 2. Step-by-step game screen (case 1). The proof so far, each line with its justification; the player has chosen Modus Tollens and lines 1 and 4 and is entering the next line. The previous step was checked and accepted.

7. Testing and edge cases

304 automated tests (python -m pytest) cover every module: a scripted solution of all five cases, play-throughs of each screen, and the samples in test_data/. A selection:

#	Input	Expected	Actual	Result
1	$P \rightarrow Q, P \therefore Q$	Valid	Valid; 1 critical row, no counterexample	Pass
2	$P \rightarrow Q, Q \therefore P$	Invalid, counterexample $P = F, Q = T$	Invalid; counterexample $P = F, Q = T$	Pass
3	Case 1: $W \rightarrow G, G \rightarrow B \therefore \neg W$; then pin evidence $\neg B$ and apply Modus Tollens twice	Invalid; then valid, both steps accepted, case solved	Invalid ($W = G = B = T$); valid; $\neg G$ and $\neg W$ accepted; solved with 100 points	Pass
4	Step check: Modus Ponens on $P \rightarrow Q$ and Q , claiming P	Rejected with a reason	Rejected: "Affirming the consequent: $P \rightarrow Q$ and Q does not prove P ."	Pass
5	Edge: no premises, $\emptyset \therefore P \vee \neg P$; contradictory premises, $P, \neg P \therefore Q$	Valid (tautology); valid only vacuously	Valid; valid with 0 critical rows, reported as vacuous	Pass
6	Edge: malformed ($P \& Q$; 60 nested brackets; a formula with 9 atoms)	A clear message each time, never a crash	"This bracket is never closed." at position 0; "nested too deeply"; "The limit is 8 atoms."	Pass
7	Edge: five $\leftrightarrow$ inside one another, on the custom case screen	Verdict shown without freezing	Verdict and truth table shown; the resolution proof is skipped with a message	Pass

8. Individual contribution

Member	Built	Files and functions
Aritra Das	Formula tree; parser with positioned error messages and size limits; formatter; evaluator; truth-table builder; validity checks. Game state with direct proof and proof by contradiction, hints, scoring, case validation, custom cases; the five case files. Repository setup.	engine/formula.py; parser.py: tokenize, parse, format_formula; evaluator.py: get_atoms, evaluate, get_subformulas; truth_table.py: build_truth_table; validity.py: classify, are_equivalent, check_argument; game/case_loader.py: validate_case, build_custom_case; game_state.py: create_game, collect_clue, assume_opposite, attempt_step; hints.py: use_hint; scoring.py: score_case; data/cases/; their tests
Prathmesh Khare	Step checker for the nine rules of inference; step-by-step CNF converter and clause generator; automatic proof by resolution refutation; missing-premise search; detection of a lying witness.	engine/rules.py: check_step; cnf.py: to_cnf, to_clauses; resolution.py: prove_by_resolution; missing_premise.py: is_consistent, try_candidates, find_liars; their tests
Ganesh Chavan	Title screen, case cabinet, case screen with stage tracker, evidence cards and step builder; custom case screen; truth table lab; styling; the backgrounds and the detective character.	app.py; ui/home.py, case_files.py, custom_case.py, truth_table_lab.py; ui/board.py: play, show_verdict; step_builder.py: show_step_builder, submit_step; tables.py: truth_table_html; board_pieces.py: proof_html; resolution_view.py: show_resolution_proof; ui/style.css; tools/make_art.py; their tests

9. Conclusion and references

Achieved. A playable detective game with five cases, in which every step of the player's proof is checked by a rule checker and the missing premise is found by truth table; a custom case screen that gives a verdict, a truth table and a resolution proof for any typed argument and lets it be played; and a truth table lab for any formula, with an equivalence check.

Limitation. The game covers propositional logic only. The truth-table method is exponential, so a case is limited to 8 atoms (256 rows), and a resolution proof that would grow too large is skipped with a message.

References. [1] K. H. Rosen, *Discrete Mathematics and Its Applications*, 8th ed., McGraw-Hill, 2019, ch. 1. [2] Streamlit documentation, docs.streamlit.io.